MODULE II

Geethu Bastian
Asst. Professor
Division of Computer Science
SOE,CUSAT

Syllabus

Map Reduce Algorithm Design: Local aggregation techniques, combiners & In-Mapper Combining, Ensuring algorithmic correctness with local aggregation, Pairs & stripes data Organization, Computing relative frequencies, secondary sorting, relational joins.

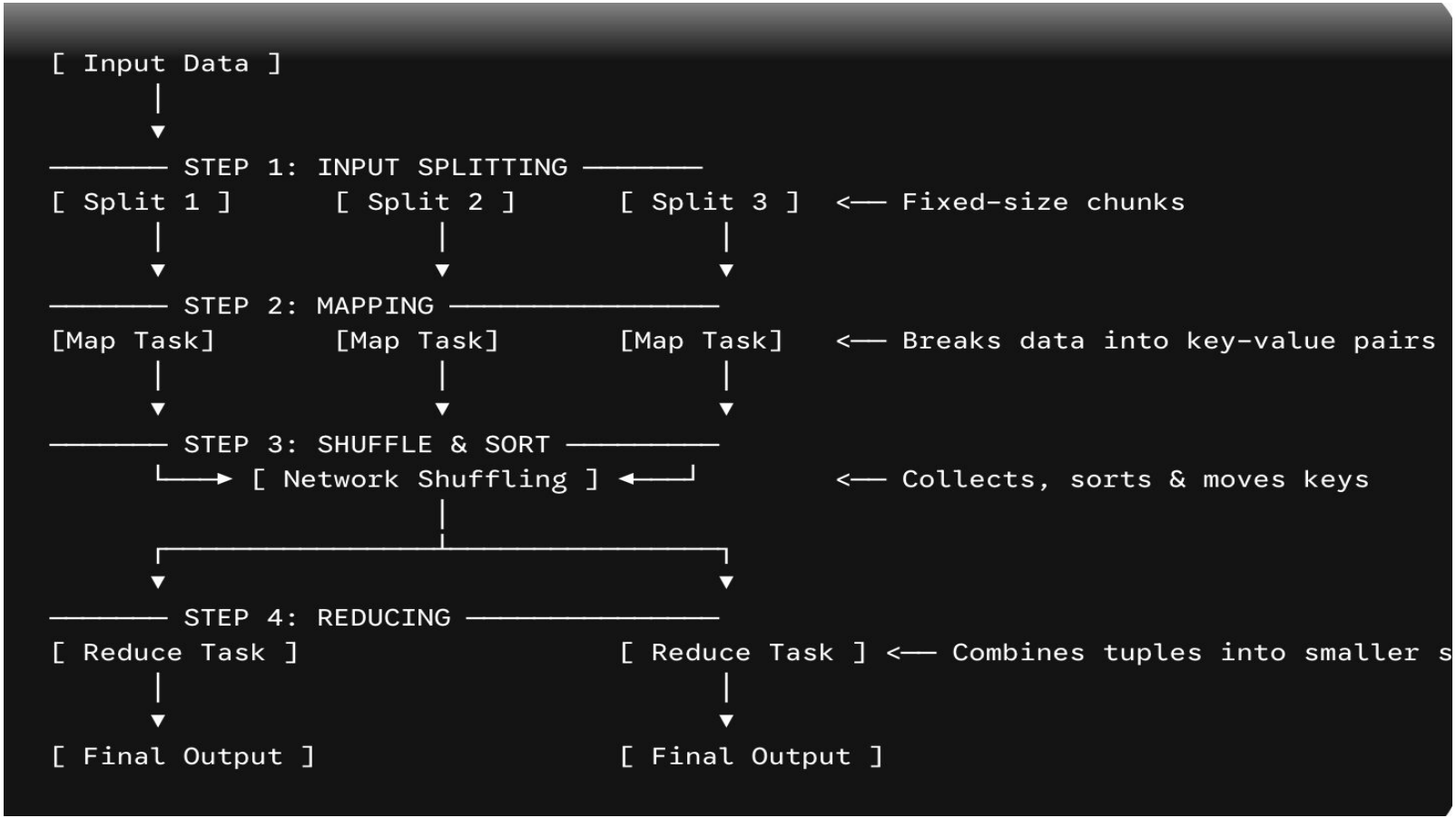
Hadoop Ecosystem: Hadoop Distributed File System (HDFS), HDFS Architecture & use cases, Hadoop cluster architecture, Need for Hadoop framework, comparison with other systems.

Hadoop Ecosystem technologies: Serialization (Avro), Coordination (ZooKeeper), Databases (HBase, Hive), Scripting (Pig), streaming (Flink, Storm), Implementing Map Reduce with Hadoop.

Map Reduce

- Map Reduce is a programming model designed to process massive data sets in parallel across a cluster.
- It abstracts away the complexities of data partitioning, task scheduling, and fault tolerance, allowing you to focus on the logic of data processing.

Core Mapreduce Workflow



Breakdown of Stages:

Input splitting: The framework divides the input data into fixed-size chunks called splits, assigning each to a Map task.

Mapping: The Map function converts input data into intermediate key-value pairs.

Shuffle & sort: A system-level phase where the framework collects, sorts by key, and shuffles intermediate data across the network so identical keys route to the same reducer.

Reducing: The Reduce function processes the grouped intermediate data and aggregates them into a smaller set of tuples

Key Design Patterns

A. Summarization

- **Use Case:** Counting words, calculating mean, or finding a maximum value.
- **Mechanism:** Uses the key as the grouping mechanism.
- **Map:** Emit (grouping_key, value).
- **Reduce:** Iterate through all values for that key and compute the final summary.

B. Filtering & Selection

- **Use Case:** When you only need a specific subset of the data.
- **Mechanism:** The entire operational logic stays inside the mapper.
- **Map:** If a record matches your criteria emit (record-id, record-content).
- **Reduce:** Often acts as a simple pass-through.

C. Joining Data (Reduce-side Join)

- **Use Case:** Joining two large datasets together (e.g., Users and Orders).
- **Map:** Tags each record with its source key. For example, setting the *User ID* as the key, and ("user", Name) as the value.
- **Reduce:** For each unique User ID, the reducer receives one user record combined with multiple corresponding order records.

- The two datasets (let's say a 10GB Users file and a 500GB Orders file) are too massive to fit into the memory of any single machine.
- MapReduce uses the network shuffle to bring records with the same Join Key (User ID) to the exact same Reducer.

Step 1: The Map Phase (Tagging)

The Mappers read both datasets simultaneously. A Mapper doesn't know if a line of data comes from Users or Orders until it parses it. Its job is to extract the **Join Key** and **tag the data** so the Reducer knows where it originated.

- **From the Users Dataset:**
  - *Input:* User_ID: 101, Name: Alice
  - *Mapper Output:* Key = 101, Value = ("User", "Alice")
- **From the Orders Dataset:**
  - *Input:* Order_ID: 999, User_ID: 101, Item: Laptop
  - *Mapper Output:* Key = 101, Value = ("Order", "Laptop")

Step 2: The Shuffle and Sort Phase (Grouping)

- This is where the MapReduce framework does the heavy lifting.
- It takes all the outputs from all Mappers, sorts them by the key (User_ID), and routes them across the network.
- Every single record across both datasets that shares User_ID: 101 will end up at the **exact same Reducer**.

Step 3: The Reduce Phase (Merging)

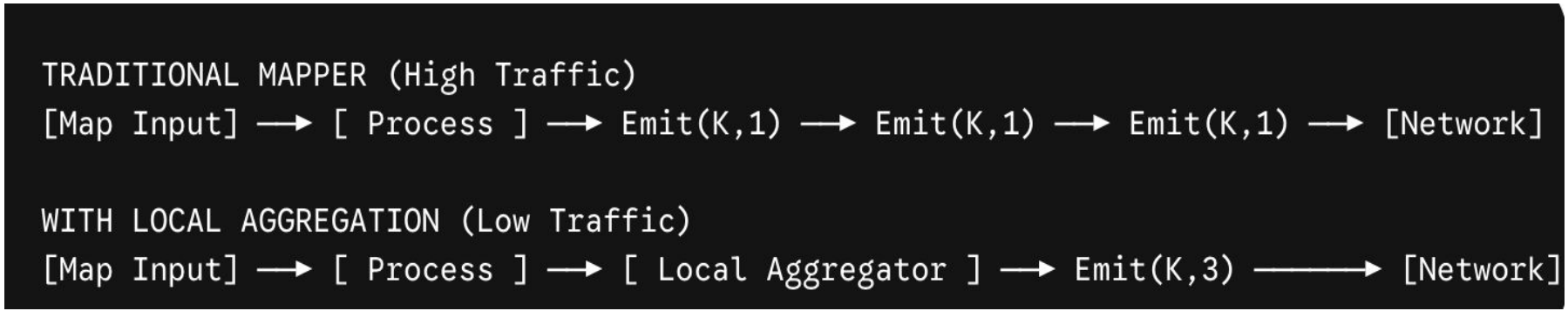
- When the Reducer for User_ID: 101 starts, it receives the key and an iterator containing *all* the values tagged by the mappers.
- The list of values arriving at the Reducer looks something like this: [("User", "Alice"), ("Order", "Laptop"), ("Order", "Book")]

Emit the Result:

- 101 -> ("Alice", "Laptop")
- 101 -> ("Alice", "Book")

Local Aggregation Techniques

- Local aggregation aims to execute as much data reduction as possible on the **map side** before sending anything over the network during the shuffle & sort phase.
- This drastically decreases network traffic and reducer workloads.



Techniques for Local Aggregation

1. Combiners (The Basic Approach)

- **Definition:** A mini-reducer running directly on the output of a Mapper node.
- **Execution:** Processes intermediate data locally on the exact node running the Map task before network emission.
- **Mathematical Constraints:** The operation **must** be associative and commutative.
  - *Works:* sum(), count(), max().
  - *Fails:* mean().
- **Note:** The framework provides no strict guarantee on how many times a combiner will be called.

Why count() Succeeds

Counting is **associative**. It doesn't matter how you group the items; the sub-counts will always add up to the total count.

Imagine you have **6 records** split across two Mappers:

- **Mapper 1** has 2 records. Its Combiner runs count() and outputs: **2**
- **Mapper 2** has 4 records. Its Combiner runs count() and outputs: **4**

When these reach the Reducer, the Reducer simply adds the sub-counts together:

2 + 4 = 6

The answer (6) is exactly correct. The framework can call the combiner as many or as few times as it wants, and the final sum will never change.

Why mean() Fails

Calculating a mean (average) is **not associative**. You cannot calculate the average of multiple averages and expect to get the correct global average, *unless every single group happens to have the exact same number of items.*

Let's look at the exact same **6 records** with values: 10, 10, 2, 2, 2, 2. The true global average is:

$\{10 + 10 + 2 + 2 + 2 + 2\}/\{6\} = \{28\}/\{6\} = 4.67$

Now let's see what happens if a Combiner tries to calculate the mean() locally on the Mappers:

- **Mapper 1 (2 records):** Has the values 10, 10.
  - *Combiner calculates local mean:* {10 + 10}/{2} = 10
- **Mapper 2 (4 records):** Has the values 2, 2, 2, 2.
  - *Combiner calculates local mean:* {2 + 2 + 2 + 2}/{4} = 2

The Combiners emit these intermediate values (10 and 2) to the Reducer.

When the Reducer receives them, it tries to compute the final average of these two incoming numbers:

$\{10 + 2\}/\{2\} = 6$

The Result:

- **Combiner's Calculation:** 6
- **True Mathematical Average:** 4.67

Because the groups had different sizes (weights), the intermediate averages distorted the final result.

The "Fix" for Mean

To compute a mean using a Combiner, you cannot actually compute the *mean* inside the Combiner. Instead, the Combiner must output a **composite value** containing both the partial sum and the partial count as a pair: (sum, count).

- **Mapper 1 Combiner outputs:** (20, 2)
- **Mapper 2 Combiner outputs:** (8, 4)
- **Reducer receives:** [(20, 2), (8, 4)]

The Reducer can then safely sum all the totals (20 + 8 = 28), sum all the counts (2 + 4 = 6), and perform the final division (28 / 6 = 4.67) at the very end.

How it works:

- If you are counting words, a standard Mapper might emit: ("apple", 1), ("apple", 1), ("apple", 1)
- Without a Combiner, all three of those separate pairs travel over the network.
- With a Combiner, it intercepts those pairs on the local machine, aggregates them, and sends just one single record over the network: ("apple", 3)

2. In-Mapper Combining

- **Definition:** An advanced pattern giving more control than a standard combiner by managing state internally inside the Mapper object.
- **Mechanism:** Instead of emitting a key-value pair for every single record processed, partial results are kept in memory using an internal data structure(HashMap).

MapReduce Mappers have a lifecycle with three main methods:

1. **setup():** Runs once at the very start. (You initialize an empty HashMap here).
2. **map():** Runs for every single record. (Instead of emitting data, you just update the counts in your HashMap).
3. **cleanup():** Runs once at the very end. (You loop through your HashMap and emit the final aggregated totals).
4. **Output:** The final aggregated tallies are emitted exactly once per split inside the Mapper's cleanup() or close() method after all records in that split are processed.

Ensuring Algorithmic Correctness

- When implementing local aggregation (Combiners or In-Mapper Combining), you must carefully manage how intermediate data is combined before it reaches the final reduction phase.
- While these patterns drastically boost performance by minimizing network footprints, they can introduce logical or statistical errors if underlying mathematical operations (like non-commutative actions) are improperly handled.

Key Strategies for Maintaining Correctness

1. **Commutative & Associative**

- Local aggregation is safest when the operations are commutative and associative.

$a+b = b+a$

$(a+b)+c = a+(b+c)$

- Standard egs include SUM, MIN, MAX & COUNT.
- Because the Map Reduce provides no guarantee on how many times a combiner will run, the logic must produce the same sum regardless of how the input is grouped.

2. Handling Non-Associative Operations (eg: Averages)

Directly calculating an average in a combiner is is a common mistake.As the average of partial averages is not same as global average.

- **Correct approach:** Instead of emitting a single value, have both mapper & combiner emit a complex value (eg a pair consisting of the sum & count).
- **Final step:** The global average is only computed once in the final reducer by dividing the total sum by the total count.

3. Type Matching

- For combiners to work correctly, its o/p key-value types must exactly match the i/p key values expected by reducer.
- Since the reducer's i/p is the Combiner's o/p, any mismatch will cause the job to fail or produce corrupted data.

4. In-Mapper Combining

- Unlike standard combiners, in-mapper combining involves manually managing state within mapper's memory across multiple records.
- Correctness with this approach is risky because the developer must manually handle memory allocation & ensure all cached data is emitted before the mapper task finishes (typically in the cleanup() method).

Pairs and Stripes Data Organization

- Imagine we are tasked with counting every time two words appear near each other in the entire library of the world. This is the co-occurrence Matrix problem (N*N).
- Since the data is too big for one computer, we use Mapreduce.
- The difference between Pairs & stripes is simply how you package data before sending it over the network.

The Pairs Approach

In this method, every single co-occurrence is treated as a unique, independent event.

- **Map task:** For every word 'w' in a sentence, you look at its neighbors 'u' and emit a pair ((w, u), 1).
- **Shuffle/sort:** The system must sort & move every single one of these pairs across the network to the right "bucket" (Reducer).
- **The Reducer Task:** The reducer receives all instances of (w, u) and sums the 1s.

The Technical Trade-off:

- **Memory Friendly:** The Mapper doesn't need to remember anything. It sees a pair, it yells it out & moves to next.
- **Network Heavy:** It creates a massive no. of Keys. If you have 4 billion words, you might be shuffling 10 billion pairs. This creates a network bottleneck.

The Stripes Approach

- Instead of sending every pair individually, you group all neighbors of a specific word into an "associative array" ("a stripe") before sending it.
- **The Map task:** For a word w, you look at all its neighbors in a document. You build a mini dictionary (Map) in your computer's memory.
  {neighbor 1: 3, neighbor 2: 1, neighbor 3: 5}
  Once you finish the document, you emit (w, {neighbor 1: 3, ….}).
- **The Shuffle/sort:** The system only moves one key pair per unique word found on that document.
- **The Reduce Task:** The Reducer receives several maps for word w and performs an element-wise sum (adding the counts for each neighbor).

The Technical Trade-off:

- **Network efficient:** You send far fewer messages over the network. You have done pre-aggregation in the mapper.
- **Memory risky:** The Mapper must keep those dictionaries in its RAM! If a word like "the" has 50,000 different neighbors, that stripe might be huge. For large documents, it can cause the program to crash (out of memory error).

Conceptual Example

Sentence: *"Data is power, knowledge is power."*

```
PAIRS METHOD:
(power, is) ⟶ 1  ]
(power, knowledge) ⟶ 1  ] ⟶ 3 Separate Network Messages Sent
(power, is) ⟶ 1  ]

STRIPES METHOD:
power ⟶ { is: 2, knowledge: 1 }  ⟶ 1 Combined Network Message Sent
```

Analogy: Pairs is like sending 100 individual letters to one person. Stripes is putting all those letters into one single box before mailing it. The box is more efficient to move.

Computing Relative Frequencies

- This is the logical next step after counting co-occurrence. The formula for the relative frequency is:
-

$$f(w_j|w_i) = \frac{N(w_i, w_j)}{\sum_k N(w_i, w_k)}$$

- Where N(w_i, w_j) is the number of times the specific pair (w_i, w_j) occurred. The denominator is the total count of all words that co-occurred with w_i (the marginal count).

Computing with Stripes

- This approach makes calculating relative frequencies very straightforward because all the data needed for the denominator is already present in the same object.
- **Reduce phase:** The reducer receives a key w_i & a list of stripes (maps) from different mappers.
- **Aggregation:** First, perform an element-wise map sum of all maps to get a final aggregated map.
  Eg: power -> {is: 10, knowledge: 5, data: 5}

- **Summation:** Iterate through the final map & sum all final values to find the total: (10 + 5 + 5) = 20.
- **Division:** Divide each element's count by the total to get the frequency.
  Result: power -> {is: 0.5, knowledge: 0.25, data: 0.25}
- Since the entire "row" of the matrix is in one place, the calculation is a simple in-memory loop.

2. Counting with Pairs

- In the pairs approach, the Reducer receives a pair (w_i, w_j) and its count, but it doesn't know the total count for w_i without looking at all other pairs starting with w_i.
- To solve this, we use a technique called **Order Inversion**.

A. The special "star" key: The mapper is modified to emit a special "star" key for every pair. For the pair (power, knowledge), the mapper emits:

- ((power, knowledge), 1)
- ((power, *), 1) —>t*his tracks the total for the word ‘power’*

B. Custom Partitioning & Sorting:

1. **Partitioner:** You must ensure all the keys with the same first word (eg, power) go to the same Reducer, regardless of the second word or the star.
2. **Sorting:** Must ensure that the (w_i, *) key arrives at the reducer before any specific pairs (w_i, w_j).

C. The Reduce Phase

1. initialize the reducer encounters (power, *) first. It sums all the "star" values to get the total margin count (the denominator) and saves it in a variable.
2. Process: when the reducer then receives (power, knowledge), it divides that count by the marginal total it just calculated first.
3. Reset: when the reducer sees a new word (eg: data) it resets the marginal counter

Secondary Sorting

- **Secondary sorting** is a technique in MapReduce that allows values associated with the same key to be **sorted before they are sent to the reducer**.
- Secondary sorting is a technique used in distributed data processing frameworks (like MapReduce and Spark) to control the order in which values for a specific key reach the reducer.
- While MapReduce guarantees that keys are sorted, the values associated with those keys arrive in an arbitrary order.
- Secondary sorting allows you to sort those values without loading them all into the computer's memory, which prevents system crashes when the dataset is too large.

Student	Subject	Marks
Alice	Math	85
Alice	Science	90
Alice	English	80
Bob	Math	70
Bob	English	75

You want the reducer to receive:

Alice:

English 80

Math 85

Science 90

Bob:

English 75

Math 70

sorted by marks:

Alice:

Science 90

Math 85

English 80

A normal MapReduce job **cannot guarantee this order** because it only sorts by the primary key (Student).

Secondary sorting solves this problem.

The Three Pillars of Secondary Sorting

1. **The Composite Key:** Instead of a single value (like a single word), we create a composite key that consists of two parts:

- **Natural Key:** The primary data that we are grouping by (e.g., word_i).
- **Secondary Key:** The value we want to sort the group by (e.g., word_j).

Example Mapper Output:

(Alice, English) → 80

(Alice, Math) → 85

(Alice, Science) → 90

(Bob, English) → 75

(Bob, Math) → 70

2.**Custom Partitioner:**

- Normally, the system hashes the entire key to decide which reducer gets the data.
- If we hashed the whole composite key, might go to entirely different reducers.
- A custom partitioner forces the system to only look at the **Natural Key** (word_i) when routing, ensuring all data starting with "word_i" ends up at the exact same reducer.
- Here partition only using the primary key (Student).

  Eg: hash(Student) % reducers

  Alice → Reducer 1

  Bob → Reducer 2

3. Custom Comparator (Grouping & Sorting):

1. **Sort Comparator:** Defines the overall framework order. We configure it to sort by the Natural Key first, and then by the Secondary Key.

MapReduce sorts the keys:

(Alice, English)

(Alice, Math)

(Alice, Science)

1. **Grouping Comparator:** Tells the reducer where one logical "group" ends and the next begins.

   Alice:

   (Alice, English)

   (Alice, Math)

   (Alice, Science)

The reducer should receive **one group per student**, not one group per composite key.

So reducer gets:

Key = Alice

Values:

English 80

Math 85

Science 90

Relational Joins

- In MapReduce, joins are primarily "data movement" problems.
- Because data is distributed across many different machines, the main challenge is moving records that share the same join key to the same physical machine at the same time.

1. **Reduce-Side Join (Repartition Join)**

- This is a highly flexible join strategy because it works regardless of the size of the datasets.
- It relies heavily on the framework's shuffle and sort phase to bring related data together.

How it works

1. Map Phase: Mappers read from two different data sources (e.g., a users table and an orders table). For every record, the mapper emits the Join Key (e.g., User ID) as the output key.

2. Tagging: To distinguish which table a record originated from, the mapper tags the value payload:

- Source A: (User ID, (Tag - User, Name))
- Source B: (User ID, (Tag - Order, Order))

3. Shuffle/Sort: The framework automatically groups all values from both sources by the common User ID.

4. Reduce Phase: The reducer receives a single User ID alongside a combined list of values from both tables. It iterates through the list, separates the "User" data from the "Order" data, and performs a cross-product to output the joined result.

2. Map-Side Join

This is a massive optimization that skips the expensive shuffle and sort phases entirely. However, it only works if one of the datasets is small enough to fit completely into memory. The mapper must have access to **both datasets**.

- **Setup:** Before the job starts, the small table (often a dimension table) is distributed (broadcast) to every single node in the cluster.
- **Map Phase:** Each mapper loads this small table into an in-memory Hash Map. As the mapper streams through the large table, it looks up the Join Key.

- **The Join:** The mapper looks up the large table's join key in its local, in-memory Hash Map. If a match is found, it emits the joined record immediately.
- **No Reduce Phase:** Because the join is resolved entirely during the map phase, a reduce phase is not required.

Hadoop Ecosystem: Hadoop Distributed File System (HDFS), HDFS Architecture & use cases, Hadoop cluster architecture, Need for Hadoop framework, comparison with other systems.

What is HDFS?

HDFS is a **distributed file system** in Hadoop that stores large files by dividing them into blocks and distributing those blocks across multiple machines while maintaining multiple copies for fault tolerance.

Hadoop Distributed File System (HDFS)

1. Storage component of Hadoop.

2. Distributed File System.

3. Modeled after Google File System.

4. Optimized for high throughput (HDFS leverages large block size and moves computation where data is stored).

5. You can replicate a file for a configured number of times, which is tolerant in terms of both software and hardware.

6. Re-replicates data blocks automatically on nodes that have failed.

7. You can realize the power of HDFS when you perform read or write on large files (gigabytes and larger).

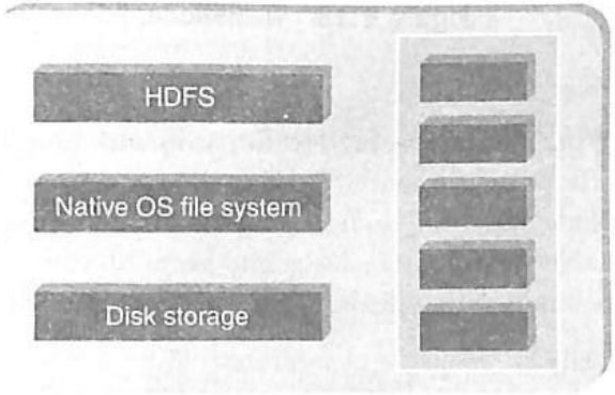


Figure 5.13 Hadoop Distributed File System.



Figure 5.14 Hadoop Distributed File System – key points.

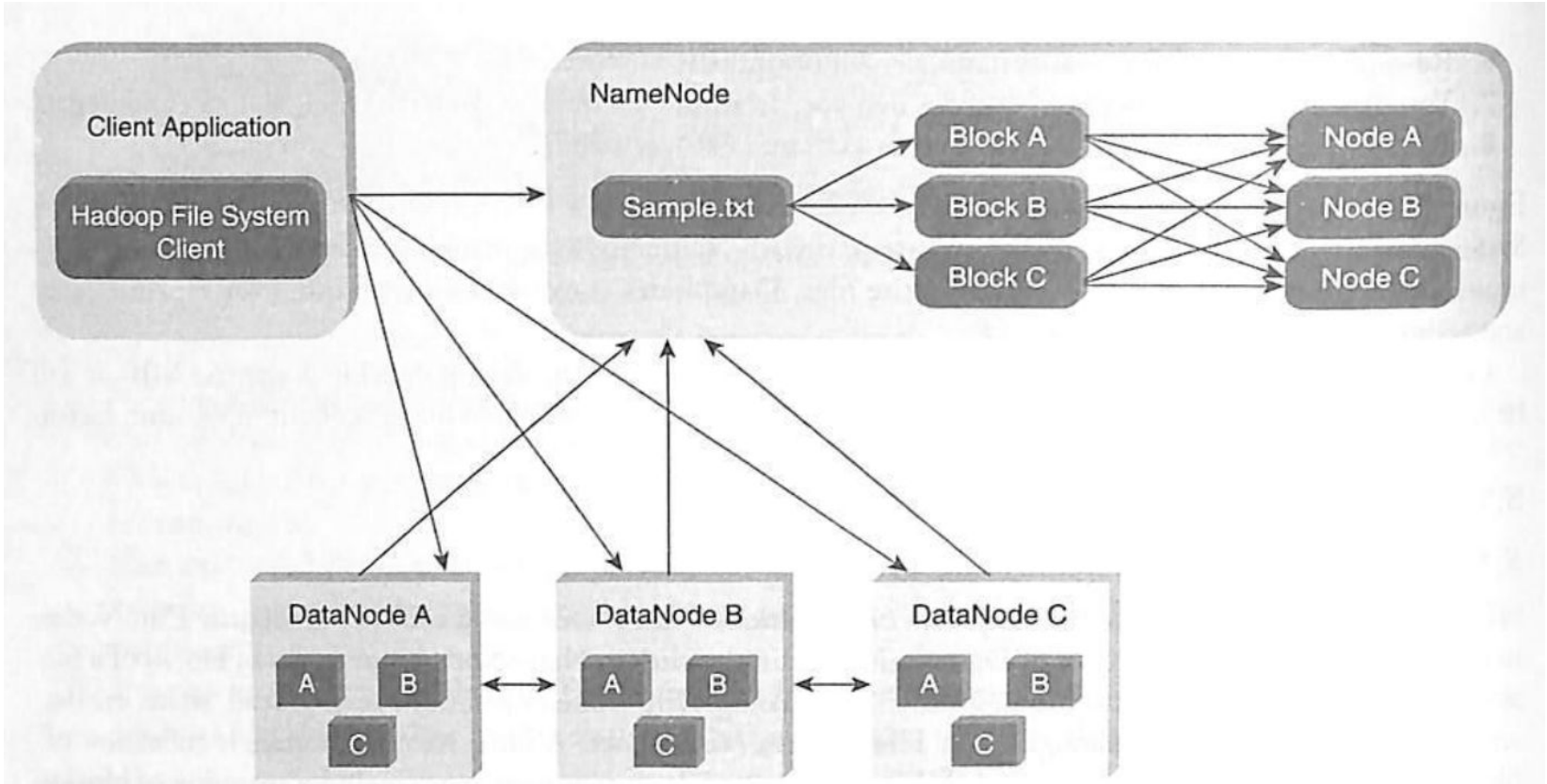


Figure 5.15 Hadoop Distributed File System Architecture.

- Client Application interacts with NameNode for metadata related activities and communicates with DataNodes to read and write files.
- DataNodes converse with each other for pipeline reads and writes.
- Let us assume that the file "Sample.txt" is of size 192 MB.
- As per the default data block size (64 MB), it will be split into three blocks and replicated across the nodes on the cluster based on the default replication factor.

HDFS Daemons

NameNode

- HDFS breaks a large file into smaller pieces called blocks.
- NameNode uses a rack ID to identify DataNodes in the rack.
- A rack is a collection of DataNodes within the cluster.
- NameNode keeps tracks of blocks of a file as it is placed on various DataNodes.
- NameNode manages file-related operations such as read, write, create,and delete. Its main job is managing the File System Namespace.
- A file system namespace is collection of files in the cluster.
- NameNode stores HDFS namespace. File system namespace includes mapping of blocks to file, file properties and is stored in a file called FsImage. NameNode uses an EditLog (transaction log) to record every transaction that happens to the file system metadata.



Figure 5.16 NameNode.

DataNode

- There are multiple DataNodes per cluster. During Pipeline read and write DataNodes communicate with each other.
- A DataNode also continuously sends "heartbeat" message to NameNode to ensure the connectivity between the NameNode and DataNode.
- In case there is no heartbeat from a DataNode, the NameNode replicates that DataNode within the cluster and keeps on running as if nothing had happened.

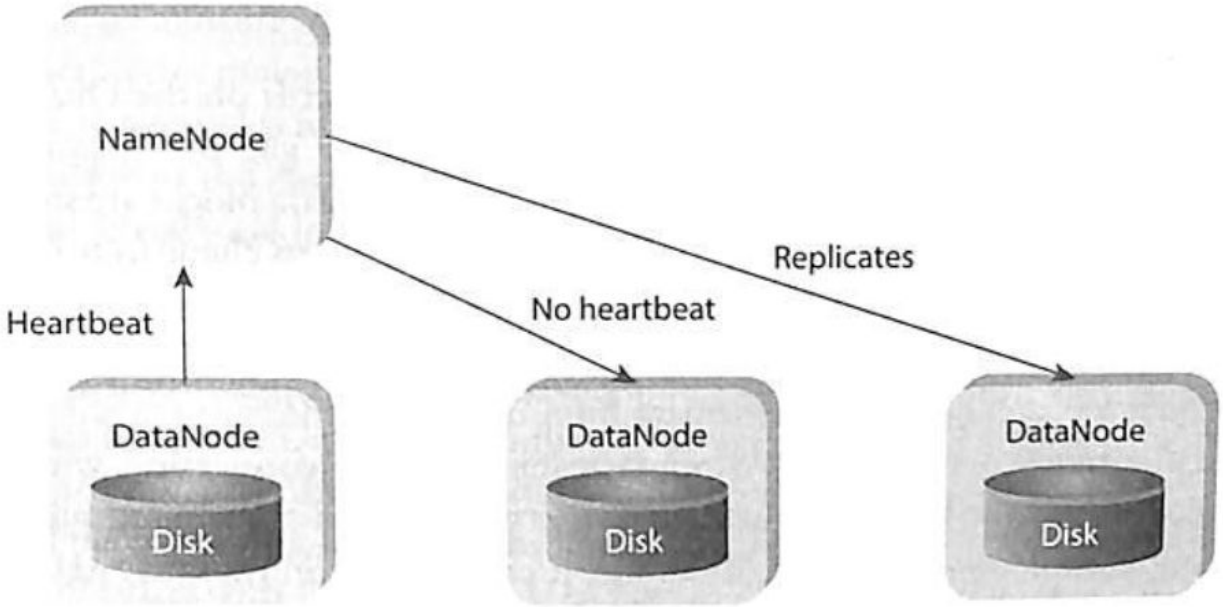


Figure 5.17 NameNode and DataNode Communication.

Secondary NameNode

- The Secondary NameNode takes a snapshot of HDFS metadata at intervals specified in the Hadoop configuration.
- Since the memory requirements of Secondary NameNode are the same as NameNode, it is better to run NameNode and Secondary NameNode on different machines.
- In case of failure of the NameNode, the Secondary NameNode can be configured manually to bring up the cluster.
- The Secondary NameNode does not record any real-time changes that happen to the HDFS metadata.

Anatomy of File Read

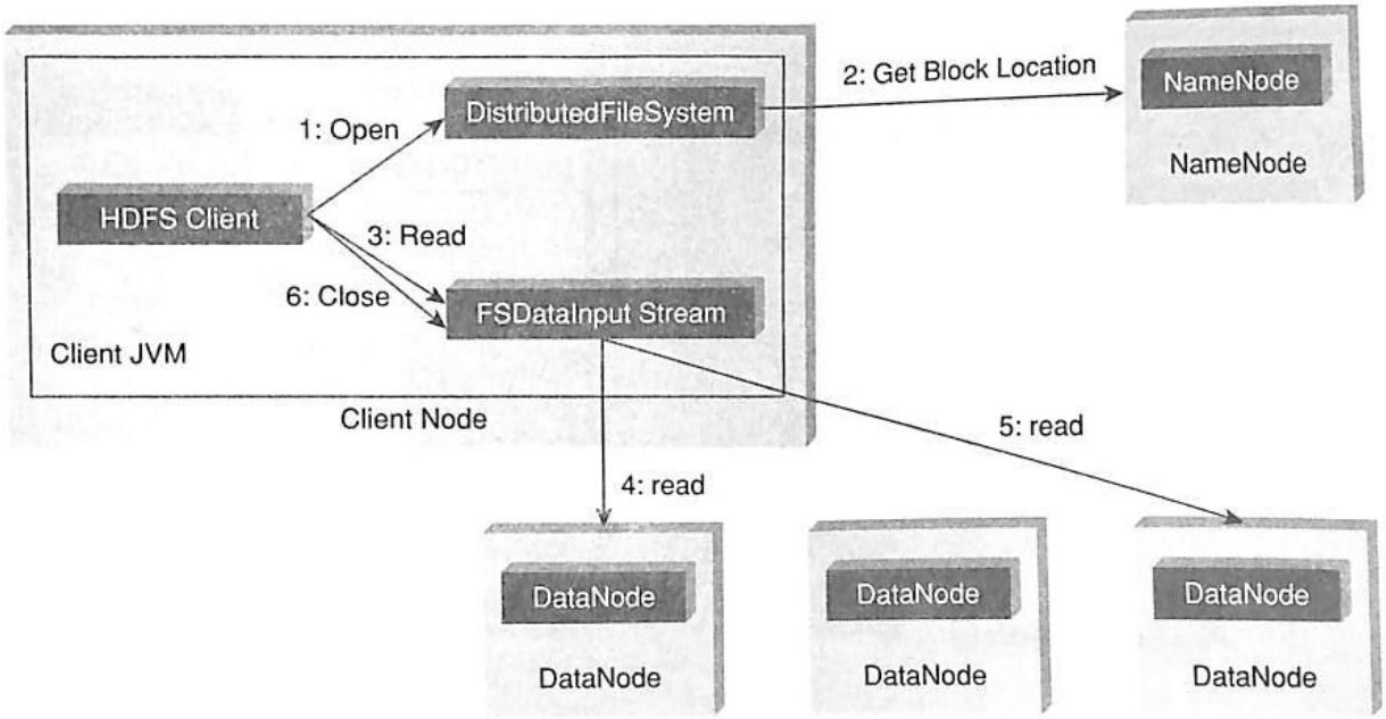


Figure 5.18 File Read.

Steps

1. The client opens the file that it wishes to read from by calling open() on the Distributed File System.

2. DistributedFileSystem communicates with the NameNode to get the location of data blocks. NameNode returns with the addresses of the DataNodes that the data blocks are stored on. Subsequent to this, the DistributedFileSystem returns an FSDatainputStream to client to read from the file.

3. Client then calls read() on the stream DFSInputStream, which has addresses of the DataNodes for the first few blocks of the file, connects to the closest DataNode for the first block in the file.

4. Client calls read() repeatedly to stream the data from the DataNode.

5. When end of the block is reached, DFSInputStream closes the connection with the DataNode. It repeats the steps to find the best DataNode for the next block and subsequent blocks.

6. When the client completes the reading of the file, it calls close() on the FSDatainputStream to close the connection.

Anatomy of File Write

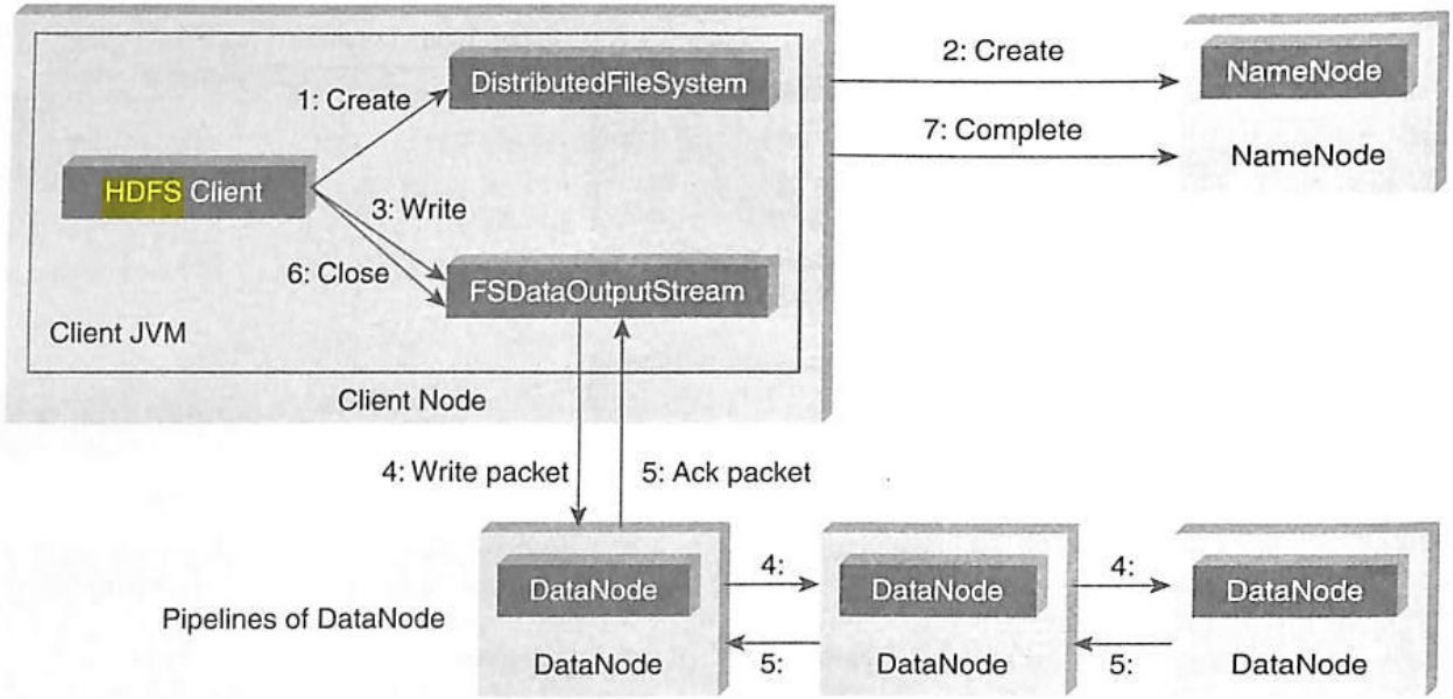


Figure 5.19 File Write.

Anatomy of File Write

1. The client calls create() on Distributed File System to create a file.

2. An RPC call to the NameNode happens through the DistributedFileSystem to create a new file. The NameNode performs various checks to create a new file (checks whether such a file exists or not). Initially, the NameNode creates a file without associating any data blocks to the file.

The Distributed File System returns an FSDataOutputStream to the client to perform write.

3. As the client writes data, data is split into packets by DFSOutputStream, which is then written to an internal queue, called data queue. DataStreamer consumes the data queue.

The DataStreamer requests the NameNode to allocate new blocks by selecting a list of suitable DataNodes to store replicas.This list of DataNodes makes a pipeline. Here, we will go with the default replication factor of three, so there will be three nodes in the pipeline for the first block.

4. DataStreamer streams the packets to the first DataNode in the pipeline. It stores packet and forwards it to the second DataNode in the pipeline. In the same way, the second DataNode stores the packet and forwards it to the third DataNode in the pipeline.

5. In addition to the internal queue, DFSOutputStream also manages an "Ack queue" of packets that are waiting for the acknowledgement by DataNodes. A packet is removed from the "Ack queue" only if it is acknowledged by all the DataNodes in the pipeline.

6. When the client finishes writing the file, it calls close() on the stream.

7. This flushes all the remaining packets to the DataNode pipeline and waits for relevant acknowledgments before communicating with the NameNode to inform the client that the creation of the file is complete.

Hadoop Default Replica Placement Strategy

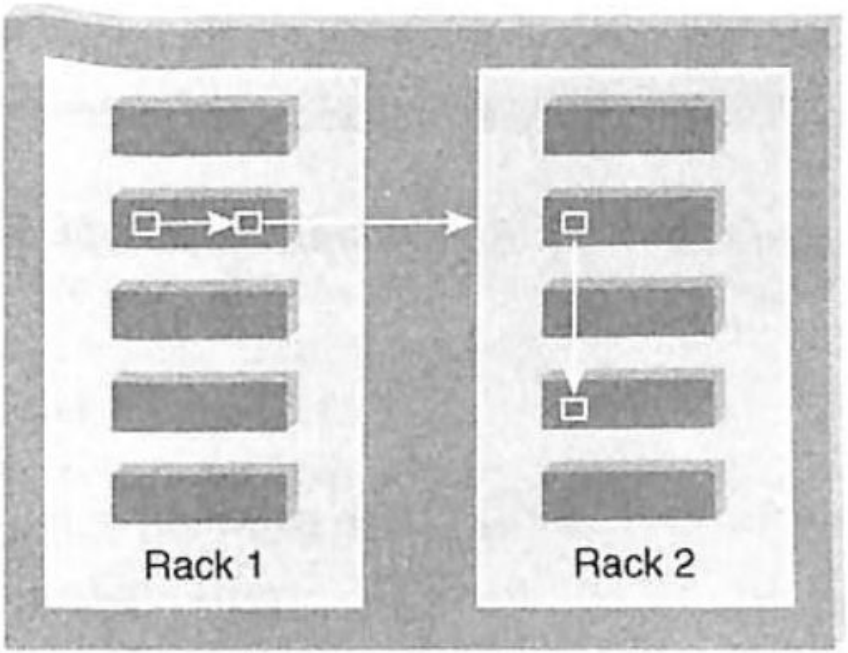


Figure 5.20 Replica Placement Strategy.

Working with HDFS Commands

1.Objective: To get the list of directories and files at the root of HDFS.

Act: hadoop fs -ls /

2.Objective: To get the list of complete directories and files of HDFS.

Act: hadoop fs -ls -R /

3.Objective: To create a directory (say, sample) in HDFS.

Act: hadoop fs -mkdir / sample

4.Objective: To copy a file from local file system to HDFS.

hadoop fs -put /root/sample/test.txt /sample/test.txt

hadoop fs -put: The command to upload files

/root/sample/test.txt: The exact location of the file on your local Linux machine.

/sample/test.txt The directory inside HDFS where you want to store.

5.Objective: To copy a file from HDFS to local file system.

Act: hadoop fs -get /sample/test.txt /root/sample/testsample.txt

-get: The sub-command that downloads the file from HDFS.

/sample/test.txt: The **source** path on HDFS.

6. Objective: To display the contents of an HDFS file on console.

Act: hadoop fs -cat /sample/test.txt

7.Objective: To copy a file from one directory to another on HDFS.

hadoop fs -cp /sample/test.txt /sample1/

-cp: The sub-command used to copy files *within* HDFS.

/sample/test.txt: The **source** file path on HDFS.

/sample1/: The **destination** directory on HDFS.

8.Objective: To remove a directory from HDFS.

hadoop fs -rm -r /sample1

-rm: The sub-command to remove files or directories.

-r: The **recursive** flag, which is mandatory when deleting a directory because it tells Hadoop to delete everything inside the folder before deleting the folder itself.

/sample1: The target HDFS directory you want to wipe out.

Special Features of HDFS

1. **Data Replication**: There is absolutely no need for a client application to track all blocks. It directs the client to the nearest replica to ensure high performance.

2. **Data Pipeline**: A client application writes a block to the first DataNode in the pipeline. Then this Data Node takes over and forwards the data to the next node in the pipeline.

This process continues for all the data blocks, and subsequently all the replicas are written to the disk.

Hadoop cluster architecture

- At its core, **Apache Hadoop** uses a distributed **Master-Slave topology** designed to store and process massive datasets (from terabytes to petabytes) across clusters of commodity hardware.
- Instead of relying on a single, expensive supercomputer, a Hadoop cluster scales horizontally by combining cheap, independent computers (nodes) that communicate over a local network.
- The modern architecture (Hadoop 2.x/3.x) splits responsibilities into three distinct pillars: **HDFS** (Storage), **YARN** (Resource Management), and **MapReduce** (Processing).

1. Storage Layer: HDFS (Hadoop Distributed File System)

HDFS handles the distributed storage by breaking large files down into fixed-size blocks (typically 128 MB) and scattering them across the cluster. It operates on a Master-Slave design:

- **NameNode (Master):** The "brain" of HDFS. It manages the file system namespace, regulates client access, and maintains the metadata (the "table of contents" showing exactly which blocks belong to which file and where they sit).
- **DataNodes (Slaves/Workers):** These daemons run on the worker machines. Their job is to store the actual data blocks and perform block creation, deletion, and replication as directed by the NameNode.
- **Fault Tolerance (Replication):** By default, HDFS automatically replicates every block 3 times across different machines and racks (Rack Awareness). If a DataNode crashes, the NameNode detects the failure via missed heartbeats and recovers the data from a surviving replica.

2. Resource Management Layer: YARN (Yet Another Resource Negotiator)

YARN is essentially the operating system of the cluster. It separates resource allocation from the processing engine, allowing Hadoop to run multiple frameworks simultaneously (like MapReduce, Apache Spark, and Flink).

- **ResourceManager (Master):** The ultimate authority that allocates CPU and memory (organized into abstract units called **Containers**) across all competing applications in the system.
- **NodeManager (Slaves/Workers):** Runs on each individual worker node, tracking the health and resource utilization (CPU/RAM) of its local containers and reporting back to the ResourceManager.
- **ApplicationMaster (Per-Application):** A temporary coordinator spawned for each specific job. It negotiates resource containers from the ResourceManager and collaborates with the NodeManagers to execute and monitor tasks.

3. Processing Layer: MapReduce

MapReduce is the native, software framework used to write applications that process vast amounts of data in parallel. It relies heavily on **Data Locality**—bringing the computation code directly to the physical node where the HDFS data block lives, minimizing expensive network traffic.

- **Map Phase:** The framework splits the input data into independent chunks processed by map tasks in parallel, producing intermediate key-value pairs.
- **Shuffle & Sort:** The framework groups all intermediate data by key so that all identical keys end up at the exact same reducer.
- **Reduce Phase:** The reducer aggregates, filters, or processes the combined values to yield the final output.

Comparison with other systems.

	Hadoop	RDBMS
Data Variety	Used for structured, semi-structured and unstructured data. Hadoop supports a variety of data formats in real time such as XML, JSON, and text-based flat file formats.	Used for structured data
Data Storage	Usually datasets of size terabytes, petabytes	Usually datasets of size gigabytes
Querying	HiveQL	SQL
Query Response	In Hadoop, there is latency due to batch processing.	In RDBMS, query response time is immediate.
Schema	Schema required on read	Schema required on write
Speed	Writes are faster compared to reads as there is no adherence to schema required at the time of inserting or writing data. Schema is enforced at read time	Reads are very fast (supported by building indexes on required columns).
	Hadoop is designed for write once read many times. It does not work for random reading and writing of a few records like RDBMS.	RDBMS is designed for read and write many times.

Cost	Apache Hadoop is open-source, large-scale, distributed, scalable, data intensive computing.	Available as proprietary RDBMS such as Oracle, MS SQL Server, IBM DB2, etc. Also open-source RDBMS are available such as MySQL, PostgreSQL, etc.
Use Cases	Analytics, data discovery	OLTP (Online Transaction Processing). Mainly used to store and process day-to-day business data.

	Hadoop	RDBMS
Throughput	High	Low
Scalability	Horizontal (Hadoop scales by adding nodes to a Hadoop cluster of easily available commodity machines).	Vertical: RDBMS scales vertically by increasing the horsepower (CPU, Hard Disk Capacity, RAM, etc.) of the machine.
Hardware	Commodity/Utility Hardware	High End Servers
Integrity	Low	High. Obeys ACID properties A – Atomicity C – Consistency I – Integrity D – Durability

Hadoop Ecosystem technologies: Serialization (Avro), Coordination (ZooKeeper), Databases (HBase, Hive), Scripting (Pig), streaming (Flink, Storm), Implementing Map Reduce with Hadoop.

Apache Avro (Serialization)

- In distributed systems like Hadoop, data must be converted into a byte stream to be sent over the network or saved to disk.
- This process is called **serialization** (and the reverse is **deserialization**).
- While Java has built-in serialization, it is often bloated and slow.
- Apache Avro is a popular open-source data serialization system designed specifically for big data processing.
- It relies heavily on a schema to make data exchange fast, compact, and highly efficient.

Key Characteristics of Avro

Schema Deployment: Unlike JSON, where every single record repeats the keys (e.g., "name": "Alice"), Avro stores the schema separately. The actual data stream contains only the raw values, making it incredibly compact.

Self-Describing Data: The schema is written directly into the file's header. Anyone reading the file later can read the header first to understand exactly how to parse the binary data.

Rich Data Structures: It supports primitive types (int, long, string, boolean) as well as complex types (records, enums, arrays, maps, unions).

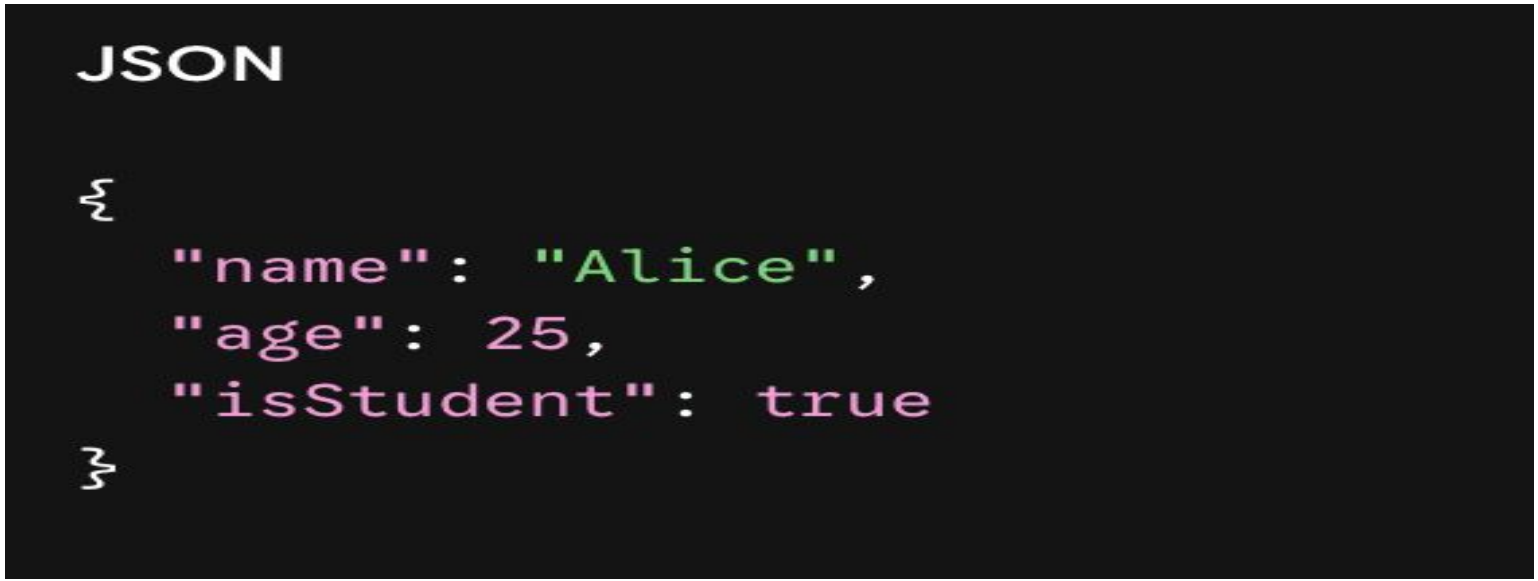
No Code Generation Required: Avro allows you to read and write data dynamically using just the schema file.

The Avro Architecture: Schema + Binary

Avro splits data management into two distinct parts:

1. **The Human Side (The Schema):** A readable JSON text layout (usually with a .avro extension) that defines the data types and structures.
2. **The Machine Side (The Binary System):** When you serialize a record, Avro converts the values into a compact, unreadable binary payload. Its size is a tiny fraction of an equivalent JSON object.

What a standard JSON Object looks like: An object (JavaScript Object Notation) is enclosed in curly braces {}, and the data is written as key: value pairs separated by commas.



Schema Evolution

- Avro supports **Schema Evolution**, meaning it gracefully handles data structures changing over time.
- Avro handles schema changes like missing fields,added fields, and changed fields.
- It allows a system to read old data with a new schema, or new data with an old schema, fully supporting both backward and forward compatibility.

Apache ZooKeeper (Coordination)

- In a distributed system like Hadoop, hundreds of independent computers try to work together.
- Because it is a "shared-nothing" architecture, these machines do not natively know what the others are doing.
- Apache ZooKeeper is a centralized, highly reliable coordination service built specifically to solve coordination problems.
-

- ZooKeeper uses data nodes called **znodes** arranged in a hierarchical tree structure.
- Every znode can store a small amount of data and have child znodes.
- There are two critical types of znodes:
- **Persistent Znodes:** These stay alive until they are explicitly deleted. They are typically used for storing permanent cluster configurations (e.g., /config/database to store shared database credentials).
- **Ephemeral Znodes:** These are temporary. They only exist as long as the client machine that created them stays connected to ZooKeeper. If the client disconnects or crashes, ZooKeeper automatically deletes the node within seconds

- We need Zookeeper because otherwise if the machines communicate directly to decide upon a master or if a machine is dead, will create a massive way of network traffic and also chance to ruin data.
- So Zookeeper acts as the single source of truth.

Stream Processing (Storm vs. Flink)

- In Big Data, there are two primary ways to process information:
- **Batch Processing:** Waiting for data to pile up over time before processing it together.
- **Stream Processing:** Processing data the exact second it arrives.
- Apache Storm and Apache Flink are two major frameworks built for processing high-velocity, real-time data streams.

1. Apache Storm

One of the first open-source, distributed, real-time computation systems. It is highly optimized for ultra-low latency.

- **Architecture (Spouts & Bolts):**
  - **Spouts:** The source of the stream. A spout connects to a raw data source and pumps data into the application processing graph (topology) as discrete pieces of data called **tuples**.
  - **Bolts:** The processing units. Bolts receive tuples from spouts or other bolts. They filter, aggregate, transform data, connect to databases, and emit new tuples.

Trade-offs of Storm:

- **Ultra-Low Latency:** It processes tuples one-by-one immediately.
- **At-Least-Once Processing:** Early versions of Storm guaranteed that every single record would be processed at least once, but if a server crashed, data could accidentally be repeated.

2. Apache Flink

- Unlike Storm, Flink treats data state in real time with extreme precision.
- Flink sits in the middle of a constant stream of data and allows you to run complex business logic in real time.
- **When to use Flink:** Use Flink if you need real-time data processing but require absolute accuracy and advanced calculations over time.
- **Core Principle:** Everything is a stream. Flink can handle both real-time live data and historical batch data using the exact same logic.
- **Common Use Cases:**
- **Fraud Detection:** Flink's excellent in-memory state management makes it a premier choice for identifying fraudulent patterns instantly.
- **Real-Time Analytics:** Running complex metrics dynamically as data moves.

Databases (HBase, Hive),

HBase

- It stores data in HDFS. It is the first non-batch component of the Hadoop Ecosystem.
- It is a database on top of HDFS.
- It provides a quick random access to the stored data.
- It has very low latency compared to HDFS.
- It is a NoSQL database, is non-relational and is a column-oriented database.
- A table can have thousands of columns. A table can have multiple rows. Each row can have several column families.
- Each column family can have several columns. Each column can have several key values. It is based on Google BigTable.

Difference between HBase and Hadoop/HDFS

1. HDFS is the file system whereas HBase is a Hadoop database. It is like NTFS and MySQL.

2. HDFS is WORM (Write once and read multiple times or many times). Latest versions support appending of data but this feature is rarely used. However, HBase supports real-time random read and write.

3. HDFS is based on Google File System (GFS) whereas HBase is based on Google Big Table.

4. HDFS supports only full table scan or partition table scan. Hbase supports random small range scan or table scan.

5. Performance of Hive on HDFS is relatively very good but for HBase it becomes 4-5 times slower.

6. The access to data is via MapReduce job only in HDFS whereas in HBase the access is via Java APIs, Rest, Avro, Thrift APls.

7. HDFS does not support dynamic storage owing to its rigid structure whereas HBase supports dynamic storage.

8. HDFS has high latency operations whereas HBase has low latency operations.

9. HDFS is most suitable for batch analytics whereas HBase is for real-time analytics.

Hive

- Hive: Hive is a data warehouse software project built on top of Hadoop.
- Three main tasks performed by Hive are **summarization, querying and analysis**.
- It supports queries written in a language called HQL or HiveQL which is a declarative SQL-like language.
- It converts the SQL-style queries into MapReduce jobs which are then executed on the Hadoop platform.

Difference between Hive and RDBMS

- Both Hive and traditional databases such as MySQL, MS SQL Server,PostgreSQL support SQL interface.
- However, Hive is better known as a data warehouse (D/W) rather than a database.

Scripting (Pig)

Pig: It is a high-level scripting language used wich Hadoop.

- It serves as an alternative to MapReduce.

It has two parts:

(a) Pig Latin: It is SQL-like scripting language. Pig Latin scripts are translated into MapReduce jobs which can then run on YARN and process data in the HDFS cluster. It was initially developed by Yahoo. It is immensely popular with developers who are not comfortable with MapReduce.

How it works? There is a "Load" command available to load the data from "HDFS" into Pig.

- Then one can perform functions such as grouping, filtering, sorting, joining etc.
- The processed or computed data can then be either displayed on screen or placed back into HDFS.
- It gives you a platform for building data flow for ETL (Extract, Transform and Load), processing and analyzing huge data sets.

(b) Pig runtime: It is the runtime environment.

Implementing Map Reduce with Hadoop.

- MapReduce Programming is a software framework.
- MapReduce Programming helps you to process massive amounts of data in parallel.
- In MapReduce Programming, the input dataset is split into independent chunks. Map tasks process these independent chunks completely in a parallel manner.
- The output produced by the map tasks serves as intermediate data and is stored on the local disk of that server.
- The output of the mappers are automatically shuffled and sorted by the framework.
- MapReduce Framework sorts the output based on keys.
- This sorted output becomes the input to the reduce tasks.
- Reduce task provides reduced output by combining the output of the various mappers.
- MapReduce framework also takes care of the other tasks such as scheduling, monitoring, re-executing failed tasks, etc.

- Hadoop Distributed File System and MapReduce Framework run on the same set of nodes. This configuration allows effective scheduling of tasks on the nodes where data is present (Data Locality).
- This in turn results in very high throughput.

MapReduce Daemons

1. JobTracker: It provides connectivity between Hadoop and your application. When you submit code to cluster, Job Tracker creates the execution plan by deciding which task to assign to which node.

- It also monitors all the running tasks. When a task fails, it automatically re-schedules the task to a different node after a predefined number of retries.
- JobTracker is a master daemon responsible for executing overall MapReduce job. There is a single JobTracker per Hadoop cluster.

2. TaskTracker: This daemon is responsible for executing individual tasks that is assigned by the Job Tracker.

- There is a single Task Tracker per slave and spawns multiple Java Virtual Machines (JVMs) to handle multiple map or reduce tasks in parallel.
- Task Tracker continuously sends heartbeat message to JobTracker.
- When the Job Tracker fails to receive a heartbeat from a TaskTracker, the Job Tracker assumes that the Task Tracker has failed and resubmits the task to another available node in the cluster.

Once the client submits a job to the Job Tracker, it partitions and assigns diverse MapReduce tasks for each Task Tracker in the cluster.

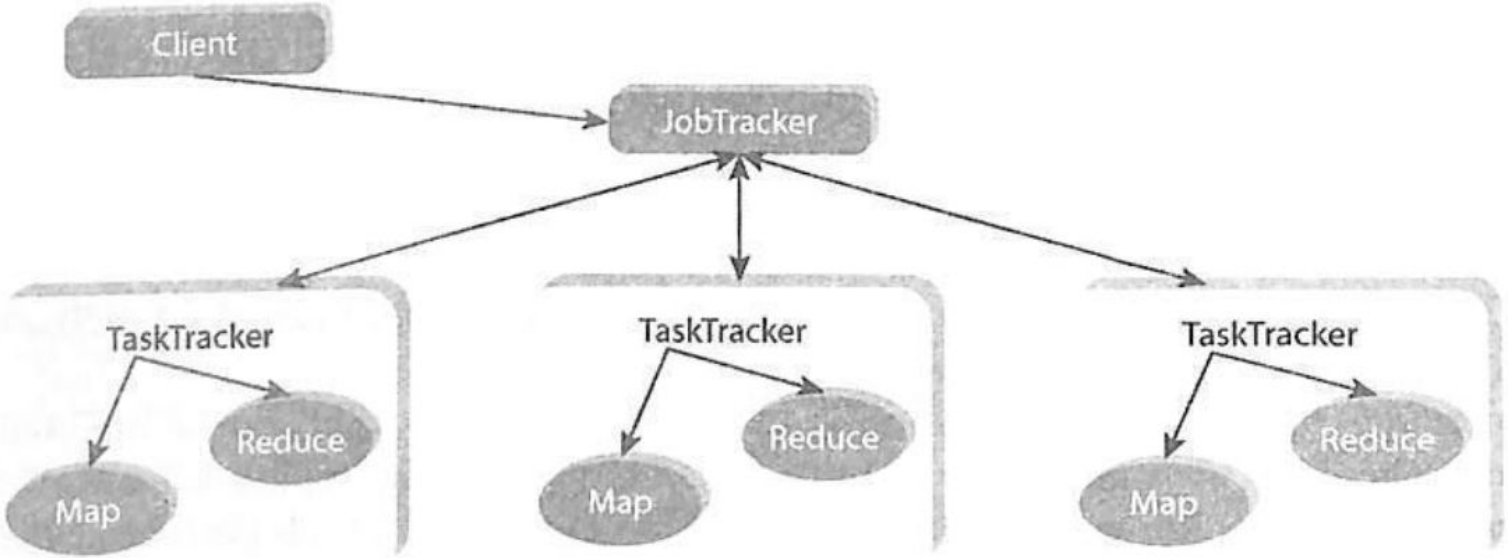


Figure 5.22 JobTracker and TaskTracker interaction.

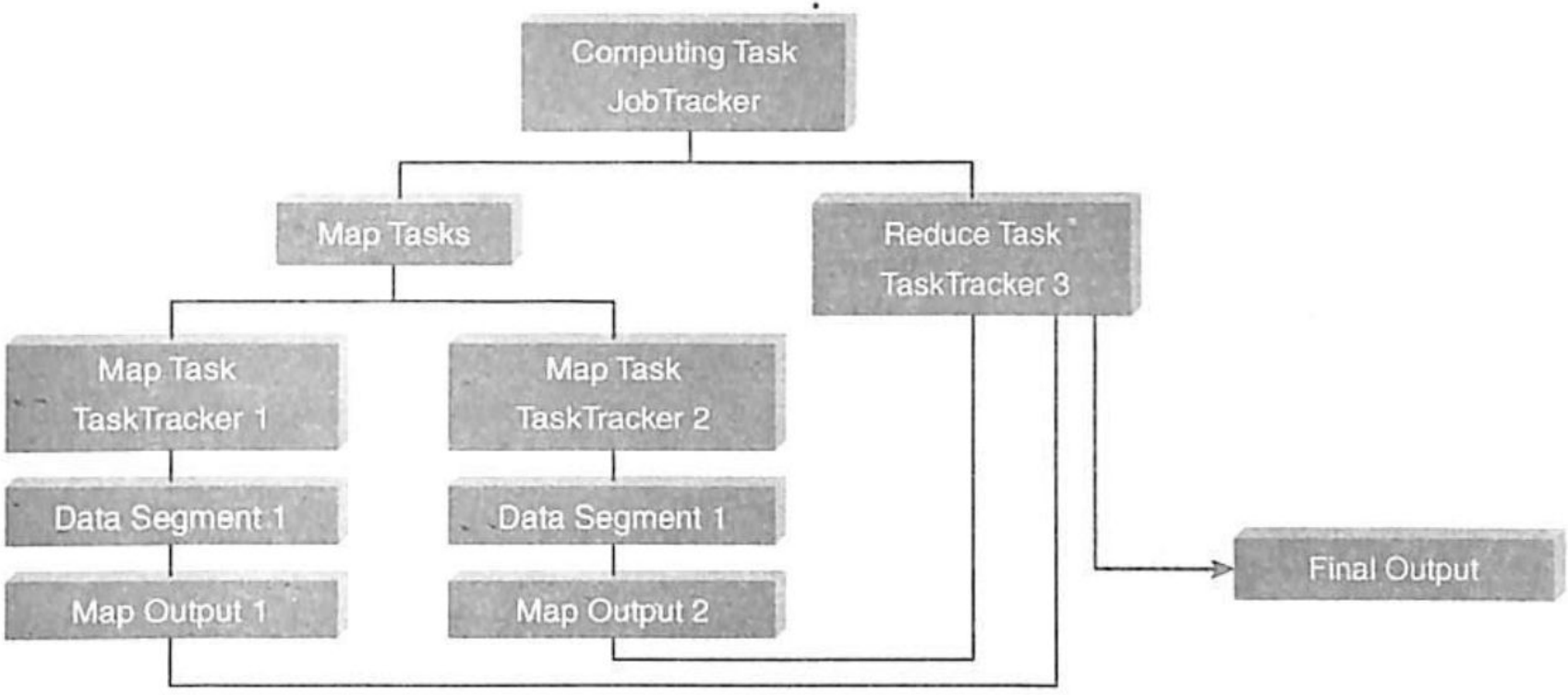
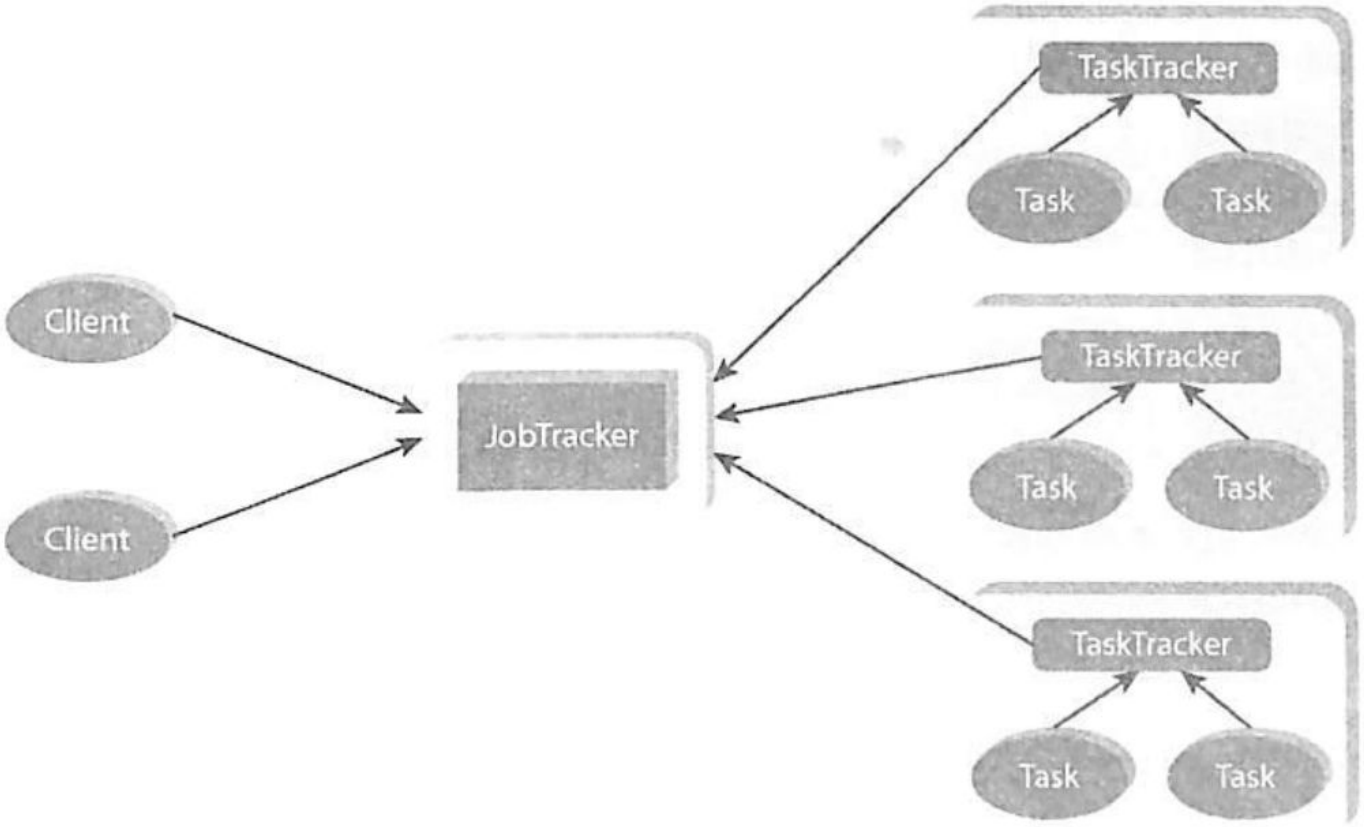


Figure 5.23 MapReduce programming workflow.

MapReduce programming architecture.



Steps

1. First, the input dataset is split into multiple pieces of data (several small subsets).

2. Next, the framework creates a master and several workers processes and executes the worker processes remotely.

3. Several map tasks work simultaneously and read pieces of data that were assigned to each map task. The map worker uses the map function to extract only those data that are present on their server and generates key/value pair for the extracted data.

4. Map worker uses partitioner function to divide the data into regions. Partitioner decides which reducer should get the output of the specified mapper.

5. When the map workers complete their work, the master instructs the reduce workers to begin their work. The reduce workers in turn contact the map workers to get the key/value data for their partition.

The data thus received is shuffled and sorted as per keys.

6. Then it calls reduce function for every unique key. This function writes the output to the file.

7. When all the reduce workers complete their work, the master transfers the control to the user program.


Thank
you